

Día D — qué ejecuta cada uno

Ramas y archivos ya están decididos: no se inventan en vivo. Los comandos se copian tal cual. Lo único que se escribe a mano es **la Historia que dicte el profesor**.

Quién	Su rama	Su archivo de spec	Sus archivos de código
Sergio	main	specs/001-ai-media-generator/spec.md	ninguno — solo dicta y reparte
Mateo	feature/hu-viva-ui	specs/001-ai-media-generator/hu-viva/mateo.md	app/**/*.jsx, components/**
Johan	feature/hu-viva-api	specs/001-ai-media-generator/hu-viva/johan.md	app/api/**/*.route.js
Tomás	feature/hu-viva-datos	specs/001-ai-media-generator/hu-viva/tomas.md	lib/**, test/**
Santiago	main	ninguno	merge, build y despliegue

Paso 0 — 20 minutos antes, los cinco

```
cd ~/Documents/web/speckit-ai-generator
git checkout main
git pull origin main
npm install
npm test # debe decir 0 failed
```

Santiago, además:

```
npm run smoke https://ia-generator-
openspec.vercel.app
```

Deja el panel de Vercel abierto y la URL cargada en el celular.

Paso 1 — Sergio registra la Historia · ≈ 2 min · rama `main`

```
git checkout main
git pull origin main
```

Pega esto en el agente, con las historias literales del profesor:

```
/speckit-specify Agrega dos User Stories nuevas al final de la seccion User Scenarios de
specs/001-ai-media-generator/spec.md, numeradas User Story 5 y User Story 6.
HU 5: "historia literal que dictó el profesor"
HU 6: "segunda historia literal"
Cada una con sus Acceptance Scenarios numerados en formato Given/When/Then.
NO crees una feature nueva. NO corras create-new-feature.sh. NO cambies de rama.
NO toques plan.md ni tasks.md. Solo edita ese spec.md que ya existe.
```

```
git add specs/001-ai-media-generator/spec.md
git commit -m "HU en vivo: User Stories 5 y 6 dictadas en clase"
git push origin main
```

Después reparte en voz alta: **a Mateo lo que se ve, a Johan lo que responde la API, a Tomás lo que se guarda y se prueba**. Las ramas ya están asignadas en la tabla de arriba — no se cambian.

Paso 2 — Mateo, Johan y Tomás, cada uno en su máquina · ≈ 6 min

2.1 — Crea tu rama. Copia solo tu columna:

MATEO

```
git checkout main
git pull origin main
git checkout -b feature/hu-viva-
ui
```

JOHAN

```
git checkout main
git pull origin main
git checkout -b feature/hu-viva-
api
```

TOMÁS

```
git checkout main
git pull origin main
git checkout -b feature/hu-viva-
datos
```

2.2 — Escribe tu spec. Cambia **mateo** por **johan** o **tomas** según quién seas:

Escribe `specs/001-ai-media-generator/hu-viva/mateo.md`, y solo ese archivo.

Historia: **"la parte de la HU que me tocó, literal"**

El archivo lleva cuatro secciones:

1. Historia — la frase de arriba, textual.
2. Criterios de Aceptacion — numerados CA-1, CA-2, CA-3, en formato Dado / Cuando / Entonces.
3. Plan — la lista exacta de archivos que vas a crear o modificar, y por que cada uno.
4. Tasks — numeradas, y cada una dice que CA cubre.

Reglas: no toques `spec.md`, `plan.md` ni `tasks.md`. Solo puedes tocar **`app/**/page.jsx y components/**`**.

Si necesitas otro archivo, parate y dimelo antes de escribir.

Si la Historia guarda datos, van a MongoDB por `lib/db.js`, nunca en memoria.

2.3 — Implementa test-first (misma sustitución de nombre):

Implementa `specs/001-ai-media-generator/hu-viva/mateo.md`, test-first: por cada Criterio de Aceptacion, primero la prueba que falla y despues el codigo que la pasa.

Nombra cada prueba empezando por su criterio: `it("CA-2: ...")`.

Toca unicamente los archivos de la seccion Plan de ese documento. Ninguno mas.

Al terminar corre `npm test` y dejalo en verde.

2.4 — Prueba, commit y push. Copia solo tu columna:

MATEO

```
npm test
git add -A
git commit -m "feat: HU viva UI
(CA-1 a CA-3)"
git push -u origin feature/hu-
viva-ui
```

JOHAN

```
npm test
git add -A
git commit -m "feat: HU viva API
(CA-1 a CA-3)"
git push -u origin feature/hu-
viva-api
```

TOMÁS

```
npm test
git add -A
git commit -m "feat: HU viva
datos (CA-1 a CA-3)"
git push -u origin feature/hu-
viva-datos
```

Si `npm test` **sale rojo, no hagas push**. Dile al agente cuál CA falla, que lo arregle y vuelva a correr `npm test`. Cuando tu push termine, dílo en voz alta.

Paso 3 — Santiago integra y despliega · ≈ 2 min · rama `main`

Solo cuando los tres hayan dicho que subieron:

```
git checkout main
git pull origin main
git fetch origin
```

Un merge por línea. Espera a que termine cada uno:

```
git merge --no-ff origin/feature/hu-viva-ui
git merge --no-ff origin/feature/hu-viva-api
git merge --no-ff origin/feature/hu-viva-datos
```

Si un merge se enreda: `git merge --abort` y reintentas solo esa rama.

Con los tres adentro:

```
npm test
npm run build
git push origin main
```

Ese push despliega en Vercel solo. Mientras construye, proyecta el grafo:

```
git log --graph --oneline -15
```

Paso 4 — Todos validan sobre el despliegue · ≈ 1 min

1. **Santiago** espera el check verde del deploy en Vercel (1–2 min) y corre:

```
npm run smoke https://ia-generator-openspec.vercel.app
```

2. Todos abren `https://ia-generator-openspec.vercel.app` en el celular, con datos móviles, y recorren la Historia nueva de punta a punta.
3. Si preguntan por la trazabilidad, el dueño abre su `hu-viva/nombre.md`, señala el CA y corre esa sola prueba:

```
npm test -- -t "CA-2"
```